

Speech Emotion Recognition System

A deep learning-based application for recognizing emotions from speech audio files using CNN, BiLSTM, and Multi-Head Attention mechanisms.

Features

- **Advanced Deep Learning Model:** Combines CNN for feature extraction, Bidirectional LSTM for sequence processing, and Multi-Head Attention for improved accuracy
- **Data Augmentation:** Uses noise addition, time stretching, and pitch shifting to enhance model robustness
- **Rich Feature Extraction:** Extracts MFCC, delta coefficients, chroma, spectral contrast, rolloff, centroid, zero-crossing rate, and RMSE
- **Interactive GUI:** User-friendly interface built with Tkinter for easy audio file upload and emotion prediction
- **Prediction History:** Tracks and displays previous predictions
- **Visual Feedback:** Shows emoji representations of detected emotions

Supported Emotions

- Happy 😊
- Sad 😞
- Angry 😡
- Surprised 😲
- Neutral 😐
- Fear 😨
- Disgust 🤢

Requirements

Python Libraries

```
bash
```

```
pip install librosa
pip install numpy
pip install scikit-learn
pip install tensorflow
pip install keras
pip install resampy
pip install pillow
pip install sounddevice
pip install pydub
```

System Requirements

- Python 3.7+
- WAV audio files for prediction
- Emoji image files (happy.png, sad.png, angry.png, surprised.png, neutral.png, fear.png, disgust.png)

Dataset

This project uses the **TESS (Toronto Emotional Speech Set)** dataset.

- Dataset path should be updated in the code: `C:\Users\HP\Downloads\TESS Toronto emotional speech set data`
- The dataset contains emotional speech samples from two actresses (young and old)
- Seven emotion categories are included

Installation

1. Clone the repository:

```
bash
git clone https://github.com/yourusername/speech-emotion-recognition.git
cd speech-emotion-recognition
```

2. Install dependencies:

```
bash
pip install -r requirements.txt
```

3. Download the TESS dataset and update the path in the code
4. Place emoji images in the project directory

Usage

Training the Model

Run the Jupyter notebook or Python script:

```
bash  
  
python "project final version.py"
```

The model will:

1. Extract features from audio files
2. Apply data augmentation
3. Train the CNN-BiLSTM-Attention model
4. Save training history and evaluation metrics

Running the GUI Application

The GUI will launch automatically after training. You can:

1. **Upload Audio:** Click "Audio Prediction" and upload a WAV file
2. **View History:** Check previous predictions
3. **About:** Learn more about the application

Model Architecture

```
graph TD
    A[Input Layer] --> B[Conv1D (64 filters) + ReLU]
    B --> C[MaxPooling1D]
    C --> D[Conv1D (128 filters) + ReLU]
    D --> E[MaxPooling1D]
    E --> F[Dropout (0.5)]
    F --> G[Bidirectional LSTM (128 units)]
    G --> H[Multi-Head Attention (4 heads, key_dim=64)]
    H --> I[Layer Normalization]
```

↓
Global Average Pooling
↓
Dropout (0.5)
↓
Dense (Softmax)

Training Configuration

- **Optimizer:** Adam
- **Loss Function:** Categorical Crossentropy
- **Batch Size:** 32
- **Epochs:** 50 (with early stopping)
- **Validation Split:** 20%
- **Callbacks:** EarlyStopping, ReduceLROnPlateau

Feature Extraction

The system extracts the following audio features:

- **MFCC** (Mel-Frequency Cepstral Coefficients): 40 coefficients
- **Delta MFCC:** First and second order derivatives
- **Chroma Features:** Pitch class representation
- **Spectral Contrast:** Difference between peaks and valleys
- **Spectral Rolloff:** Frequency below which 85% of energy is contained
- **Spectral Centroid:** Center of mass of the spectrum
- **Zero Crossing Rate:** Rate of sign changes
- **RMSE:** Root Mean Square Energy

Data Augmentation

Three augmentation techniques are applied:

1. **Noise Addition:** Random Gaussian noise
2. **Time Stretching:** Speed modification (rate=0.9)
3. **Pitch Shifting:** Frequency shift (± 2 semitones)

Results

The model is evaluated using:

- Accuracy and Loss metrics
- Confusion Matrix
- Classification Report (Precision, Recall, F1-Score)

Project Structure

```
speech-emotion-recognition/  
|  
├── project final version.py    # Main script  
├── README.md                  # This file  
├── requirements.txt           # Python dependencies  
|  
├── emoji/                     # Emoji images  
|   ├── happy.png  
|   ├── sad.png  
|   ├── angry.png  
|   ├── surprised.png  
|   ├── neutral.png  
|   ├── fear.png  
|   └── disgust.png  
|  
└── TESS/                      # Dataset directory  
    ├── OAF_angry/  
    ├── OAF_disgust/  
    └── ...
```

Acknowledgments

- **University of Toronto** for the TESS dataset
- **Clevered** for guidance and support throughout the project development

Future Improvements

- ☐ Support for additional audio formats (MP3, M4A)
- ☐ Real-time emotion recognition from microphone input
- ☐ Multi-language support
- ☐ Model optimization for faster inference
- ☐ Web-based interface

License

This project is open-source and available under the MIT License.

Contact

For questions or contributions, please open an issue or submit a pull request.

Note: Make sure to update the dataset path and ensure all emoji images are present before running the application.